



CS3014 - Estructuras de Datos Avanzadas

Notas de clase 01

Dynamic Graphs - Heavy-light Decomposition

8 de junio de
2026

Prof. Víctor Racsó Galván Oyola

1. Descomposiciones de árboles

Hasta ahora hemos tenido algunas ideas para poder realizar consultas específicas (Euler Tour para LCA) o consultas/modificaciones en subárboles (DFS Timestamps) de manera eficiente.

Sin embargo, también es posible recibir consultas/modificaciones sobre las aristas/nodos del camino entre un par de nodos.

Para ello, podemos usar una descomposición de árboles llamada *heavy-light decomposition*.

1.1. Heavy-Light Decomposition

Esta descomposición define 2 tipos de arista: pesada (heavy) y liviana (light), de manera que concatenaremos las pesadas en caminos y las livianas estarán libres. Siempre se define el significado de arista pesada y las livianas son las aristas restantes.

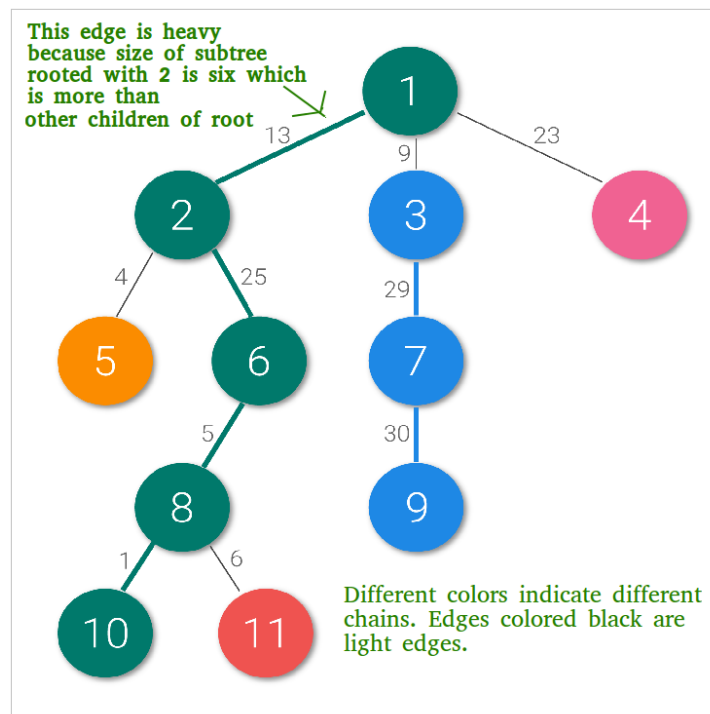


Figura 1: Heavy-Light Decomposition

Hay 2 definiciones de ambos tipos de arista, lo cual no cambia la complejidad de las operaciones pero sí la estructura de los caminos. Veremos ambas y analizaremos sus complejidades

respectivas.

1.1.1. Mayor que la mitad

- **Arista pesada:** Aquella arista (u, v) tal que el tamaño del subárbol de v es mayor que la mitad el tamaño del subárbol de u . ($2\text{subtree}[v] > \text{subtree}[u]$).

1.1.2. Mayor que sus hermanos

- **Arista pesada:** Aquella arista (u, v) tal que el tamaño del subárbol de v es mayor que los tamaños de los subárboles de sus hermanos w , si hay algún empate, se considera solo una como pesada. ($\text{subtree}[v] \geq \text{subtree}[w], \forall w \in \text{children}[u] \setminus \{v\}$).

Proposición 1

En cualquiera de las dos definiciones, al bajar por una arista liviana (u, v) se cumple que $\text{subtree}[v] \leq \frac{\text{subtree}[u]}{2}$.

Demostración. 1. Mayor que la mitad: Una arista liviana está definida como (u, v) tal que $2\text{subtree}[v] \leq \text{subtree}[u] \rightarrow \text{subtree}[v] \leq \frac{\text{subtree}[u]}{2}$.

2. Mayor que sus hermanos: Una arista liviana (u, v) existe si u tiene al menos 2 hijos, por lo tanto $\text{subtree}[v] \leq \text{subtree}[w]$, donde w es el hijo de la arista pesada (u, w) . Entonces, tendremos que:

$$\text{subtree}[v] \leq \text{subtree}[w] \rightarrow 2\text{subtree}[v] \leq \text{subtree}[v] + \text{subtree}[w] \leq \text{subtree}[u] - 1 < \text{subtree}[u]$$

$$2\text{subtree}[v] < \text{subtree}[u]$$

$$\text{subtree}[v] < \frac{\text{subtree}[u]}{2}$$

□

Gracias a la prueba anterior, podemos deducir que para llegar desde cualquier nodo x hasta la raíz del árbol tendremos que pasar por un máximo de $O(\log n)$ aristas livianas y, como consecuencia, por $O(\log n)$ **caminos de aristas pesadas**.

Si usamos alguna estructura logarítmica (Segment Tree suele ser el más usado) para almacenar la información de los caminos de aristas pesadas, tendremos una complejidad de $O(\log^2 n)$ tanto para consulta como para modificación.

La implementación siguiente es de la definición "Mayor que sus hermanos", esta realiza la descomposición luego de un preprocesamiento de los tamaños de subárboles.

```

1 int T = 0;
2
3 void DFS(int u, int p = -1){
4     subtree[u] = 1;
5     for(int i = 0; i < G[u].size(); i++){
6         if(G[u][i] == p){
7             swap(G[u].back(), G[u][i]);

```

```

8     }
9     int v = G[u][i];
10    if(v == p) continue;
11    DFS(v, u);
12    if(subtree[v] > subtree[G[u][0]]){
13        swap(G[u][i], G[u][0]);
14    }
15    subtree[u] += subtree[v];
16 }
17 if(p != -1){
18     G[u].pop_back();
19 }
20 }
21
22 void HLD(int u){
23     in[u] = T++;
24     for(int v : G[u]){
25         par[v] = u;
26         nxt[v] = (v == G[u][0] ? nxt[u] : v);
27         h[v] = h[u] + 1;
28         HLD(v);
29     }
30     out[u] = T - 1;
31 }

```

Notemos que estaremos guardando algunos arreglos extra:

- *in* y *out* son los timestamps de entrada y salida según el Euler Tour.
- *nxt*[*u*] es el inicio de la cadena de *u* (si el nodo pertenece a una arista liviana, *nxt*[*u*] = *u*).
- *h* son las profundidades de cada nodo.

Propiedad 2

El rango $[in[nxt[u]], in[u]]$ contiene todos los nodos desde el inicio de la cadena del nodo *u* hasta si mismo. Esto se da porque estamos colocando al nodo de la arista pesada como primero en la lista de adyacencia.

1.1.3. Consultas

Para consultar información sobre un camino $u \rightsquigarrow v$ uno pensaría en obtener el LCA *l* de ambos nodos y luego obtener las respuestas parciales de cada subcamino $u \rightsquigarrow l$ y $v \rightsquigarrow l$; sin embargo, es más sencillo hacer la siguiente observación:

Observación 3

El LCA de *u* y *v* pertenece a una sola cadena, a la cual llegarán eventualmente los nodos *u* y *v* a medida que asciendan en el árbol. Además, esta cadena es la más alta a la que podrán llegar sin salirse del camino $u \rightsquigarrow v$.

Gracias a la observación anterior, podemos considerar el mover hacia arriba a cada nodo hasta que estos estén en la misma cadena (fácilmente identificable, pues esto se dará cuando *nxt*[*u*] = *nxt*[*v*]). Entonces, podemos aplicar el algoritmo greedy de subir lo más que se pueda a aquel que tenga *nxt* con menor *h*.

La siguiente implementación está hecha para consultar el máximo valor de algún nodo en el camino $a \rightsquigarrow b$, dado que tenemos un segment tree sobre el Euler Tour del árbol.

```

1 int path(int a, int b){
2     int res = -2e9;
3     while(nxt[a] != nxt[b]){
4         if(h[nxt[a]] < h[nxt[b]]) swap(a, b);
5         res = max(res, query(in[nxt[a]], in[a]));
6         a = par[nxt[a]];
7     }
8     if(h[a] > h[b]) swap(a, b);
9     res = max(res, query(in[a], in[b]));
10    return res;
11 }

```

Es sencillo notar que el bucle se ejecutará hasta que estén ambos nodos en la misma cadena, para lo cual realizamos una consulta extra para cubrir dicho rango de nodos. La complejidad de este algoritmo es de $O(\log^2 n)$.

1.1.4. Modificaciones

Para realizar modificaciones sobre un camino podemos considerar el mismo algoritmo de consulta pero aplicando las actualizaciones:

```

1 void changePath(int a, int b, int w){
2     while(nxt[a] != nxt[b]){
3         if(h[nxt[a]] < h[nxt[b]]) swap(a, b);
4         update(in[nxt[a]], in[a], w);
5         a = par[nxt[a]];
6     }
7     if(h[a] > h[b]) swap(a, b);
8     update(in[a], in[b], w);
9     return res;
10 }

```

Cuya complejidad será la misma de $O(\log^2 n)$. Por otro lado, si queremos modificar sobre un subárbol, podemos usar el mismo Segment Tree (si se trabaja sobre las mismas propiedades) y modificar el rango $[in[u], out[u]]$. Este segundo tipo de modificación tomará $O(\log n)$.

Podemos notar que esta forma de implementación es bastante flexible y potente.

Nota: Si las modificaciones e información son sobre las aristas y no sobre los nodos, podemos considerar que la posición del nodo u representa a la arista $(par[u], u)$ y realizar las modificaciones correspondientes a las funciones.

1.1.5. Problemas para implementar

- [Query on a tree](#)
- [Subtrees And Paths](#)
- [Can you answer these queries VII](#)
- [Queries on tree again!](#)

1.2. Extendiendo el HLD: Small to Large Trick

Con Heavy-Light Decomposition consideramos asignar tipos a las aristas, definiendo dichos tipos para que se cumplan ciertas condiciones sobre el camino desde la raíz de un árbol hasta cada nodo u .

Consideremos ahora el caso en el que se nos da un árbol T , sobre el cual nosotros queremos responder a cierta información de cada uno de sus nodos u , considerando que dicha información depende del subárbol de u .

Es importante recordar la definición de arista pesada y ligera, pues debido a esta se da que existen una cantidad de $O(\log n)$ aristas ligeras en el camino desde la raíz del árbol hasta cualquier nodo u .

Entonces, si pudiéramos procesar cada nodo dicha cantidad de aristas ligeras y una vez por cada cadena pesada, tendríamos una complejidad de $O(n \log n)$.

Es ahora que podemos plantear la siguiente idea:

Si procesamos con un DFS cada nodo, considerando que no borraremos la información del subárbol al cual accedemos mediante una arista pesada y aplicamos el mismo criterio en la recursión, el trabajo hecho sobre cada nodo es $O(\log n)$.

Para que sea más sencillo de ver, propondremos el pseudocódigo:

```

1 solve(u, keep):
2     bigChild = -1
3     for v in G[u]:
4         if v is pi[u]: continue
5         if (u, v) is light-edge:
6             solve(v, false) // Resolvemos cada hijo de arista ligera pero sin guardar
                             ↳ la información
7         else:
8             bigChild = v
9     if bigChild != -1:
10        solve(bigChild, true) // Resolvemos el hijo de arista pesada guardando la
                               ↳ información
11    addInfo(u) // Agregamos la información de todos los hijos de arista ligera y u
12    // Resolver el nodo u con la información disponible
13    if not keep:
14        removeInfo(u) // Quitamos la información de todos los hijos de u y u

```

Si analizamos el proceso, es fácil notar que:

1. Si bajamos por una arista ligera, la información es procesada 1 vez y borrada luego de ello.
2. Si bajamos por una arista pesada, la información es procesada 1 vez y se mantiene.

Debido a lo anterior, como borramos la información del nodo u por cada arista ligera, tendremos que reponerla la misma cantidad de veces, en total procesaremos la información del nodo u $O(\log n)$ veces. Finalmente, la complejidad será de $O(n \log n)$ inserciones y eliminaciones de información.

1.2.1. Implementación

Se puede plantear la implementación del pseudocódigo en base al preprocesamiento del Heavy-Light Decomposition que ya tenemos, con el cual colocábamos al hijo pesado en la primera posición de la lista de adyacencia y eliminábamos al padre.

```

1 void DFS(int u, int p = -1){
2     subtree[u] = 1;
3     for(int i = 0; i < G[u].size(); i++){
4         if(G[u][i] == p){
5             swap(G[u].back(), G[u][i]);
6         }

```

```

7     int v = G[u][i];
8     if(v == p) continue;
9     DFS(v, u);
10    if(subtree[v] > subtree[G[u][0]]){
11        swap(G[u][i], G[u][0]);
12    }
13    subtree[u] += subtree[v];
14 }
15 if(p != -1){
16     G[u].pop_back();
17 }
18 }
19
20 void add(int u, int val, int start){
21     if(val == 1) agregarInfo(u);
22     else quitarInfo(u);
23     for(int i = start; i < G[u].size(); i++){
24         add(G[u][i], val, 0); // Solo debemos restringir en el primer nivel
25     }
26 }
27
28 void solve(int u, int keep){
29     for(int i = 1; i < G[u].size(); i++){
30         solve(G[u][i], 0);
31     }
32     if(!G[u].empty()) solve(G[u][0], 1);
33     add(u, 1, 1);
34     // Resolver con la información disponible
35     if(!keep){
36         add(u, -1, 0);
37     }
38 }

```

Aprovechando la naturaleza de la lista de adyacencia, planteamos la función ‘add’ con una variable *start* que define si ignoramos o consideramos el hijo pesado, de manera que solo debemos restringir las aristas del primer nivel de descendientes (hijos directos), así que en la recursión deberemos llamar para cada hijo con *start* = 0.

1.2.2. Problemas para practicar

- [Summing in a Tree](#)
- [Lomsat gelral](#)
- [Tree Requests](#)
- [Tree and Queries](#)

Referencias